

Algorithms and Analysis Report

COSC2123/3119

Neuromancer: Hacking with Graphs

Danial Ansari (s4119075)

Assessment Type	Individual assignment. Submit online via Classroom50.
Due Date	Tuesday 2 nd April 2025, 23:59. Late penalty: 10% (3 marks) per 24-hour period.
Marks	30

I certify that this is all my own original work. If I took any parts from elsewhere, then they were non-essential parts of the assignment, and they are clearly attributed in my submission. I will show I agree to this honour code by typing "Yes": **Yes**

Task B: Minimum Spanning Tree

(4 marks, max 1 page)

B.1 — Prim's pseudo-code

(2 marks)

Algorithm 2 PRIM(G) — mst/prim.py with an indexed binary min-heap

Require: connected graph $G = (V, E)$

Ensure: minimum spanning tree T and its total firewalls

```
1:  $s \leftarrow$  first vertex of  $V$ ;  $Q \leftarrow$  empty indexed min-heap
2:  $Q.PUSH(0, s, \text{nil})$   $\triangleright O(1)$ 
3: for each  $r \in V \setminus \{s\}$  do  $\triangleright |V|-1$  times
4:    $Q.PUSH(\infty, r, \text{nil})$   $\triangleright O(1)$ :  $\infty$  never sifts up
5:  $\text{inTree} \leftarrow \emptyset$ ;  $T \leftarrow []$ ;  $\text{total} \leftarrow 0$   $\triangleright O(1)$ 
6: while  $Q$  is not empty do  $\triangleright |V|$  iterations
7:    $(f, r, p) \leftarrow Q.POP()$   $\triangleright O(\log|V|)$  sift-down
8:    $\text{inTree} \leftarrow \text{inTree} \cup \{r\}$   $\triangleright O(1)$  hash insert
9:   if  $p \neq \text{nil}$  then  $\triangleright$  every pop but the first
10:     $T.ADD((p, r, f)); \text{total} += f$   $\triangleright O(1)$ 
11:    for each  $(u, w) \in G.GETNEIGHBOURS(r)$  do
12:      if  $u \notin \text{inTree}$  and  $u \in Q$  then  $\triangleright$  list  $\Theta(\deg r)$ ; matrix  $\Theta(|V|)$ 
13:         $Q.DECREASEKEY(u, w, r)$   $\triangleright O(1)$  each;  $2|E|$  in total
14: return  $T, \text{total}$   $\triangleright O(\log|V|)$ ; once per edge
```

Total. Pushes cost $\Theta(|V|)$, the $|V|$ pops $\Theta(|V| \log |V|)$, and line 13 runs exactly once per connection (when the first of its endpoints is popped; at the second the other endpoint is already in the tree), so at most $|E|$ sift-ups of $O(\log |V|)$. The neighbour scans of line 11 total $\Theta(|V| + |E|)$ on the list and $\Theta(|V|^2)$ on the matrix. Hence $T_{Pr, list} \in \Theta(|E| \log |V|)$ and $T_{Pr, mat} \in \Theta(|V|^2 + |E| \log |V|)$ (derived in C.1).

B.2 — Greedy-delete analysis

(2 marks)

What must be checked. Greedy delete may remove the heaviest remaining connection (u, v) only if The Fold stays connected without it, that is, only if some *other* path still joins u and v . Every delete therefore needs a connectivity test on the whole remaining network.

Why this is far dearer than a cycle check. Adding a connection can only *merge* two components. Union-find tracks those merges incrementally, so Kruskal's question "would (u, v) close a cycle?" is just two FINDs, near-constant amortised. Deleting a connection can *split* a component, and union-find cannot un-merge. Nothing learned in one test survives to the next, because a later delete may destroy the path an earlier test relied on. So each test is a fresh BFS/DFS from u . In the worst case (for example when (u, v) is a bridge, so v is never found) it visits every node and every remaining connection: $\Theta(|V| + |E_{cur}|)$, where $|E_{cur}| \geq |V| - 1$ connections are still present.

Total. After the $\Theta(|E| \log |E|)$ sort, up to all $|E|$ connections are tested, one traversal each. Each test removes at most one connection, so throughout the first $|E|/2$ tests at least $|E|/2$ connections remain, and each of those tests can cost $\Theta(|V| + |E|/2)$. That alone is $|E|/2 \cdot \Theta(|V| + |E|/2) = \Omega(|E|(|E| + |V|))$, and $|E|$ tests of at most $O(|V| + |E|)$ each give the matching upper bound. Greedy delete is therefore $\Theta(|E|(|E| + |V|))$: $\Theta(|V|^4)$ on a dense Fold, against $\Theta(|V|^2 \log |V|)$ for Kruskal. Only a fully dynamic connectivity structure, far more complex than union-find, could make each test cheaper.

Task C: Choosing the Right Tools

(8 marks, max 2 pages)

C.1 — Complexity and justification

(3 marks)

Costs are read off Algorithm 1 (Kruskal) and Algorithm 2 (B.1). Union-find uses union by rank and path compression, so each FIND/UNION is near-constant. Because $|V| - 1 \leq |E| \leq |V|^2/2$, we have $\log |E| = \Theta(\log |V|)$.

Kruskal's + matrix. GET_EDGES must read every cell of the upper triangle to find the non-zero ones: $|V|(|V| - 1)/2$ reads whatever $|E|$ is (creating the $|E|$ edge objects is inside this). The rest is as in Figure 7: merge sort $|E| \log_2 |E|$, $|V|$ MAKE-SETs, two FINDs per examined edge ($\leq 2|E|$), and $|V| - 1$ UNIONS and adds. Closed form $|V|^2/2 + |E| \log_2 |E| + 2|E| + 3|V| - 2$, so

$$T_{Kr,mat} \in \Theta(|V|^2 + |E| \log |E|).$$

Prim's + list. $|V|$ pushes at $O(1)$ (priority ∞ never moves up): $\Theta(|V|)$. $|V|$ pops, each a sift-down: $\Theta(|V| \log |V|)$. GET_NEIGHBOURS walks only r 's chain, and $\sum_r \deg(r) = 2|E|$, so all scans total $\Theta(|V| + |E|)$, with two $O(1)$ hash checks per entry. DECREASEKEY is called exactly $|E|$ times, each an $O(\log |V|)$ sift-up in the worst case: $O(|E| \log |V|)$. Total $\Theta(|V| \log |V| + |E| \log |V|)$, and as $|E| \geq |V| - 1$, $T_{Pr,list} \in \Theta(|E| \log |V|)$.

Prim's + matrix. The heap work is identical, $\Theta(|V| \log |V|) + O(|E| \log |V|)$. But each GET_NEIGHBOURS scans a full row of $|V|$ cells, and it is called $|V|$ times: $\Theta(|V|^2)$ regardless of $|E|$. So $T_{Pr,mat} \in \Theta(|V|^2 + |E| \log |V|)$.

Combination	Class	Sparse, $ E = \Theta(V)$	Dense, $ E = \Theta(V ^2)$
Kruskal + list	$\Theta(E \log E)$	$\Theta(V \log V)$	$\Theta(V ^2 \log V)$
Kruskal + matrix	$\Theta(V ^2 + E \log E)$	$\Theta(V ^2)$	$\Theta(V ^2 \log V)$
Prim + list	$\Theta(E \log V)$	$\Theta(V \log V)$	$\Theta(V ^2 \log V)$
Prim + matrix	$\Theta(V ^2 + E \log V)$	$\Theta(V ^2)$	$\Theta(V ^2 \log V)$

When sparse, the list versions are a log factor *below* the matrix ones. When dense, all four share the worst-case class $\Theta(|V|^2 \log |V|)$, so constants, and how often the log terms are really paid, decide (C.3).

C.2 — Experiment: setup and results

(2.5 marks)

Varied: density d , by setting num_edges to $|E| = \text{round}(d \cdot |V|(|V| - 1)/2)$ (at least $|V| - 1$). I used 20 levels: a bare tree ($d = 0.01$), every 0.01 from 0.02 to 0.10 (where the ranking changes), every 0.1 from 0.2 to 0.9, then 0.95 and 1. **Held fixed:** $|V| = 200$, firewalls uniform on 1..100, one machine (Linux, 2 cores, Python 3.13), report and loader off. **Same network for both structures:** at each level I built 8 networks (seeds 11–88). Each was built twice from the same seed, once as a list and once as a matrix, so both hold identical connections. All four combinations were checked to return the same MST total.

Timing: only the solver call, which includes its own GET_EDGES/GET_NEIGHBOURS calls; graph construction is excluded. I used `utils/timer.py`, with one warm-up per combination. Then 15 *interleaved* rounds, each running all four combinations once in a fresh random order, so background load hits all four alike. Garbage collection was paused during each timed call. Per network I keep the **minimum** of the 15 runs (noise can only add time). Each point plotted is the median over the 8 networks; the bars show their interquartile range (mostly 1–6% of the median). A smaller check at

$|V|=400$ (7 levels, 4 networks) is summarised in C.3. To reproduce: `python initiate0peration.py --experiment example_config.json`

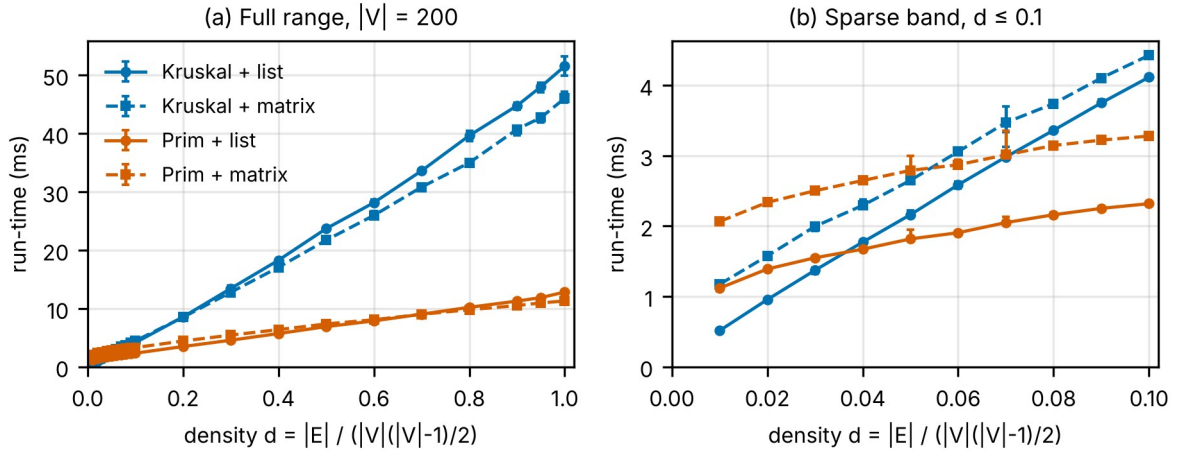


Figure 1. Solver run-time against density d for the four combinations, $|V|=200$. (a) Full range. (b) Zoom on the sparse band $d \leq 0.1$. Median over 8 networks; bars show the interquartile range.

C.3 — Interpretation and recommendation

(2.5 marks)

Kruskal (blue) curves slightly upwards. Its $|E|\log|E|$ sort dominates: from $d=0.5$ to 1, $|E|$ doubles and Kruskal + list grows $\times 2.18$, matching $2 \log(19900)/\log(9950)=2.15$. The matrix's extra $\Theta(|V|^2)$ scan shows as the gap at the tree end: 1.17 vs 0.51 ms. That gap grows $\times 4.4$ when $|V|$ doubles (2.92 ms at $|V|=400$), as $|V|^2$ predicts. From $d \approx 0.3$ the matrix version is 5–13% *faster*. Both now spend about $|V|^2$ work collecting edges, and scanning a Python row costs less per entry than walking linked-list nodes. That is a constant, not a class.

Prim (orange) is almost linear and far flatter than C.1's worst case. With `--heap-stats` I counted the work: `DECREASEKEY` is called exactly $|E|$ times, but at $d=1$ only 2.6–4.9% of calls lower a key, and those move about 1 level (not $\log_2|V| \approx 8$). So the $|E|\log|V|$ term costs about $|E|$ cheap checks in practice. Prim + list then behaves like $|V|\log|V|+|E|$ (growing $\times 1.86$ from $d=0.5$ to 1). Prim + matrix adds the fixed $|V|^2$ row-scan floor: 2.06 vs 1.12 ms at the tree end, again $\times 4.4$ at $|V|=400$.

Where the ranking changes. Kruskal + list is fastest for a near-tree (0.51 vs 1.12 ms at $d=0.01$). There it sorts only about $|V|$ edges, while Prim still pays $|V|$ heap pops. Prim + list takes over at $d \approx 0.036$, i.e. $|E| \approx 3.6|V|$; at $|V|=400$ the crossover sits at $|E| \approx 4|V|$ (a 2% tie at $d=0.02$). So it is fixed by the average degree, not by d . Prim + list leads up to $d \approx 0.7$, then Prim + matrix wins.

Sparse ($d < 0.1$): Prim + list. The list beats the matrix clearly (Prim + matrix is 1.4–1.8 $\times$ slower). Prim + list wins from $d \approx 0.04$ to 0.1, by up to 1.8 $\times$ at $|V|=200$ and 2.4 $\times$ at $|V|=400$ ($d=0.1$). Its only loss is at the near-tree end: 0.6 ms at $|V|=200$ (1.5 ms at $|V|=400$), a small absolute cost. Since we know $|V|$, the crossover $|E| \approx 4|V|$ means $d \approx 8/|V|$, so the Kruskal region shrinks as The Fold grows. Only around $d \approx 0.03$ –0.045 are the two list solvers within about 10%, which is close. If the Fold is known to be almost a tree ($|E| < 3|V|$), take Kruskal + list.

Dense ($d > 0.9$): Prim + matrix. Against Kruskal this is clear-cut: 3.9–4.6 $\times$ faster at $|V|=200$ and 4.9–5.9 $\times$ at $|V|=400$. Against Prim + list it is closer: 7–14% at $|V|=200$, still above the 2–4% spread between networks, so the gap is real. It widens to 34–43% at $|V|=400$. So the matrix is the right choice here, and more so for a larger Fold.

Task D: Load the Freighter

(6 marks, max 2 pages)

D.1 — Optimal single-limit algorithm

(1 mark)

Algorithm: dynamic programming over sub-problems (0/1 knapsack), as implemented in `load_weight_only`. Let $B(i, c)$ be the best load using only bricks $1 \dots i$ with c kg of capacity left. It stores a pair (records, weight): the most records possible, and the least weight that achieves them. “Best” compares records first; equal records go to the smaller weight.

$$B(i, c) = \begin{cases} (0, 0) & \text{if } i = 0 \\ B(i-1, c) & \text{if } w_i > c \\ \text{best} \{ B(i-1, c), B(i-1, c - w_i) + (v_i, w_i) \} & \text{otherwise} \end{cases}$$

The answer is $B(N, C)$. To recover the bricks, walk back from (N, C) : brick i was taken exactly when $B(i, c) \neq B(i-1, c)$.

Why it cannot miss a better load (induction on i). Any valid load S from bricks $1 \dots i$ within capacity c either leaves brick i out, so S is a valid load for $(i-1, c)$ and is no better than $B(i-1, c)$; or takes it, so $w_i \leq c$ and S without brick i is valid for $(i-1, c - w_i)$, giving no more than $B(i-1, c - w_i) + v_i$. The recurrence takes the better of exactly these two options, and each option is itself a valid load, so $B(i, c)$ is optimal. The same argument applied to the weight component shows it also returns the lightest of the equally valuable loads, because weights simply add.

Time. There are at most $(N+1)(C+1)$ distinct sub-problems. With memoisation each is solved once, in $O(1)$ (two look-ups, one add, one comparison). The walk-back is $O(N)$. So the worst case is $\Theta(NC)$ time and space. This is *pseudo-polynomial*: it grows with the number C , not with the length of its input. My loader works top-down from (N, C) and only solves the sub-problems that are actually reached (on `spec_config.json` with no volume limit, 46 of 147 cells; with both limits, 50 of 2,352).

D.2 — Both limits

(1.5 marks)

(a) Problem. $\max_{S \subseteq \{1 \dots N\}} \sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq C$ and $\sum_{i \in S} \omega_i \leq L$.

(b) Recurrence. The state gains a volume coordinate ℓ (litres left), and a brick may only be taken if it fits *both* limits:

$$B(i, c, \ell) = \begin{cases} (0, 0) & \text{if } i = 0 \\ B(i-1, c, \ell) & \text{if } w_i > c \text{ or } \omega_i > \ell \\ \text{best} \{ B(i-1, c, \ell), B(i-1, c - w_i, \ell - \omega_i) + (v_i, w_i) \} & \text{otherwise} \end{cases}$$

The change is needed because, in D.1, two partial loads with the same weight left are interchangeable: whatever completes one completes the other. With a volume limit they are not, since the one with more litres left can accept bricks the other cannot. A state that remembers only c would merge them and could return a load over the volume limit. Fixing it afterwards, by solving for weight and then checking volume, also fails: the best load within the volume limit need not be weight-optimal for any c . So each sub-problem must record both remaining capacities.

(c) Time. There are $(N+1)(C+1)(L+1)$ sub-problems, each $O(1)$ as before, so $\Theta(NCL)$ time and space. Each extra limit multiplies the table by the size of its capacity.

D.3 — Feasibility

(1.5 marks)

Counting one operation per sub-problem (a generous lower count; each really needs a few):

$$(N+1)(C+1)(L+1) = 101 \times 1001 \times 1001 = 101,202,101 \approx 1.0 \times 10^8 \text{ operations.}$$

At one second each that is about 1.0×10^8 s, roughly 1,170 days or 3.2 years, against a 3,600 s lockdown. It is about **28,000 times too slow**. Solving only the reachable sub-problems does not rescue it: on a real haul with $N = 100$ and $C = L = 300$, my loader still solved 56% of its table. For the optimal method to finish in time we would need $N C L \lesssim 3600$, so with $N = 100$ the product $C \cdot L$ would have to be at most 36. (Brute force is far worse: $2^{100} \approx 1.3 \times 10^{30}$ subsets.) **For the crew:** an optimal plan cannot be computed before the exits seal. Waiting for it means being caught with nothing loaded. Slyce must accept a good, possibly not optimal, load from a much faster method (D.4).

D.4 — Greedy loader

(2 marks)

Algorithm 3 GREEDYLOAD(bricks, C, L)

Require: bricks 1..N with (v_i, w_i, ω_i) ; capacities $C \geq 1, L \geq 1$

```

1: for each brick i do  $s_i \leftarrow v_i / (w_i/C + \omega_i/L)$   $\triangleright \Theta(N)$ 
2: MERGESORT bricks by  $s_i$ , largest first  $\triangleright \Theta(N \log N)$ 
3:  $S \leftarrow \emptyset$ ;  $c \leftarrow C$ ;  $l \leftarrow L$ 
4: for each brick i in that order do  $\triangleright \Theta(N)$ 
5:   if  $w_i \leq c$  and  $\omega_i \leq l$  then
6:      $S \leftarrow S \cup \{i\}$ ;  $c \leftarrow c - w_i$ ;  $l \leftarrow l - \omega_i$   $\triangleright$  never undone
7:  $b \leftarrow$  most valuable single brick that fits alone  $\triangleright \Theta(N)$ 
8: return S if  $\text{value}(S) \geq v_b$ , else {b}
```

How it works. Each brick gets a score: records per share of the freighter it uses, $s_i = v_i / (w_i/C + \omega_i/L)$. Dividing by the capacities puts kilograms and litres on the same scale (a fraction of the hold). It also weights each resource by how scarce it is: a brick that fills 30% of the volume limit costs more than one filling 30% of a loose weight limit. Bricks are then loaded best-score first. A brick that no longer fits is skipped, not a reason to stop, because a later and smaller brick may still fit. Finally, line 7 compares the result with the single most valuable brick that fits alone. This guards against a high-value brick losing out on score; in the weight-only case this “best of the two” rule is guaranteed to reach at least half the optimum. With the local test’s “greedy trap” (capacity 11 kg, scores 10, 10, 12), it takes d_3 then d_2 for 120 records, which is optimal, where greedy by value would take only d_1 for 100.

Why it is greedy. Each decision uses only local information, namely the brick’s own score and whether it fits *now*. It is made once and never revisited (line 6 is never undone). It does not explore alternatives, so it can miss the optimum. With two limits there is no general guarantee.

Time. Scores take $\Theta(N)$, the merge sort $\Theta(N \log N)$, and the scan and the best-single check $\Theta(N)$, so $\Theta(N \log N)$ in total. Unlike D.2, this does **not** depend on C or L , and it uses no table. For the mission, $N = 100$ needs about $100 \cdot \log_2(100) \approx 664$ comparisons plus about 300 other steps: under 1,000 s, well inside 3,600 s. In general it beats a lockdown of T seconds whenever $k N \log_2 N \leq T$ for a small constant k , even on hauls where the $\Theta(N C L)$ table could never be filled.